

Experiment 1: 8 Puzzle Problem:

Aim

To solve the 8-Puzzle problem using an Artificial Intelligence search algorithm to reach the goal state from a given initial state.

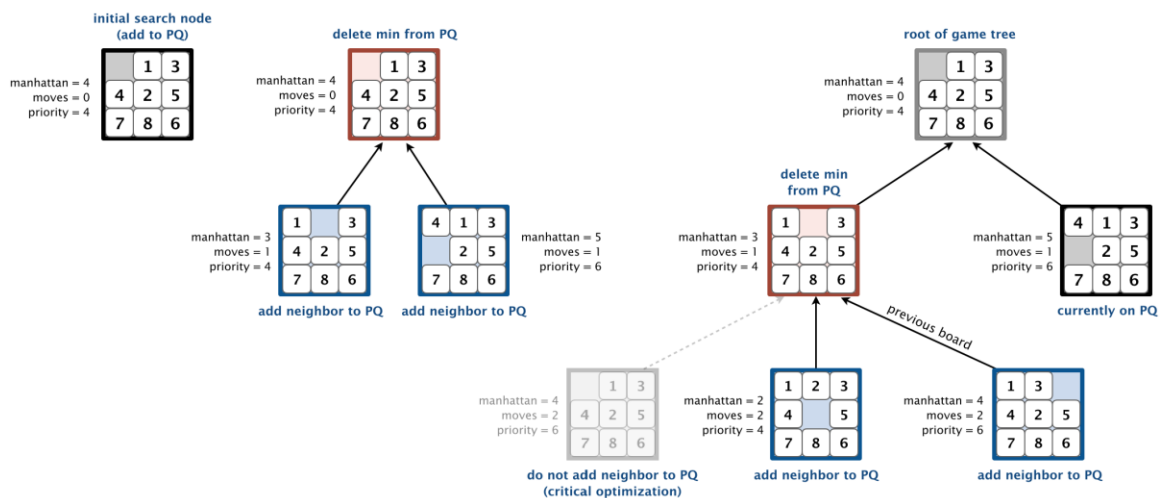
Objective

- To understand state-space search.
- To solve the 8-Puzzle problem using AI techniques.
- To find the sequence of moves from the initial state to the goal state.

Algorithm

1. Read the initial puzzle configuration.
2. Read the goal puzzle configuration.
3. Find the position of the blank tile (0).
4. Generate all possible moves (Up, Down, Left, Right).
5. Create new puzzle states by moving the blank tile.
6. Repeat until the goal state is reached.
7. Display the solution path.
8. Stop.

Flowchart



Program:

```
from collections import deque

goal=[[1,2,3],[4,5,6],[7,8,0]]

def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j

def state_to_tuple(state):
    return tuple(tuple(r) for r in state)

def bfs(start):
    queue=deque([(start,[])])
    visited=set()
    while queue:
        state,path=queue.popleft()
        if state==goal:
            return path+[state]
        visited.add(state_to_tuple(state))
        x,y=find_zero(state)
        for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx,ny=x+dx,y+dy
            if 0<=nx<3 and 0<=ny<3:
                new=[r[:] for r in state]
                new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
                if state_to_tuple(new) not in visited:
                    queue.append((new,path+[state]))

start=[]
for i in range(3):
    start.append(list(map(int,input().split()))))
print(bfs(start))
```

Sample Input

1 2 3

4 0 6

7 5 8

Sample Output

Solution Path:

[1,2,3]

[4,0,6]

[7,5,8]

...

[1,2,3]

[4,5,6]

[7,8,0]

Result

The given 8-Puzzle problem was successfully solved and the solution path was obtained.

Conclusion

The 8-Puzzle problem demonstrates state-space search in Artificial Intelligence and shows how search algorithms reach the goal state efficiently.

```

import heapq
goal=[]
print("Enter the elements of the matrix:")
for _ in range(3):
    row=list(map(int,input().split()))
    goal.append(row)
moves=[(0, 1), (1, 0), (0, -1), (-1, 0)]
def heuristic(state):
    h=0
    for i in range(3):
        for j in range(3):
            if state[i][j]!=0 and state[i][j]!=goal[i][j]:
                h+=1
    return h

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return (i,j)

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def a_star(start):
    pq=[]
    heapq.heappush(pq,(heuristic(start),0,start,[]))
    visited=set()
    while pq:
        f,g,current,path=heapq.heappop(pq)

        if current == goal:
            return path+[current]

        state_key=state_to_tuple(current)
        if state_key in visited:
            continue
        visited.add(state_key)

        x,y=find_blank(current)
        for dx,dy in moves:
            nx,ny = x+dx,y+dy

            if 0<=nx<3 and 0<=ny<3:
                new_state=[row[:] for row in current]
                new_state[x][y],new_state[nx][ny]=new_state[nx][ny],new_state[x][y]
                new_path=path+[new_state]

                if state_to_tuple(new_state) not in visited:
                    heapq.heappush(pq,(g+1+heuristic(new_state),g+1,new_state,new_path+[current]))

start=[[1,2,3],[4,0,6],[7,5,8]]
solution=a_star(start)
if solution:
    print("Solution found:")
    for step in solution:
        for row in step:
            print(row)
        print()
else:
    print("No solution found.")

```

```
PS C:\Users\athav\AppData\Local\Programs\Microsoft VS Code> & C:\Users\athav\AppData\Local\Programs\Python\Python313\python.exe c:/Users/athav/1.py
Enter the elements of the matrix:
1 2 3
4 5 6
7 8 0
Solution found:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

PS C:\Users\athav\AppData\Local\Programs\Microsoft VS Code> & C:\Users\athav\AppData\Local\Programs\Python\Python313\python.exe c:/Users/athav/1.py
Enter the elements of the matrix:
█
```